BUILDING AI AGENT

1️⃣ What is LangChain ?

LangChain is an **open-source framework** used to build applications powered by **Large Language Models (LLMs)**.

It helps developers connect LLMs with:

- APIs
- Databases
- External Tools
- Memory
- Agents
- Documents
- Search engines
- Custom business logic

LLM alone can only generate text.

LangChain helps turn an LLM into a **real application**.

🧠 Real World Analogy

Think of LLM as:

🧠 Smart Brain

LangChain as:

🛠️ Body + Hands + Memory + Internet + Tools

Without LangChain:

> LLM can answer only from trained data

With LangChain:

> LLM can search web, use tools, query DB, remember chat, automate tasks

---

🔥 Why LangChain is Used

1. Build Chatbots

Example:

- Customer support bot
- HR assistant
- Internal company assistant

---

2. Build AI Agents

Example:

User asks:

> Book flight and check weather

LangChain agent can:

- Call flight API
- Call weather API
- Compare prices
- Reply

---

3. Build RAG Apps (Retrieval Augmented Generation)

Example:

> Upload PDF.

Ask:

> Summarize this legal contract

LangChain can:

- Read PDF
- Split text
- Search relevant chunks
- Ask LLM
- Return accurate answer

---

4. Connect Multiple Tools

Example:

LLM + SQL DB + Email + Slack + APIs

🧩 Core Components of LangChain

1. Models

Connect OpenAI, Ollama, Claude etc.

Example:

ChatOllama()

ChatOpenAI()

---

2. Prompt Templates

Reusable prompts.

Example:

Explain {topic} simply

---

3. Chains

Sequence of tasks.

Example:

User Query → Prompt → LLM → Output

---

4. Memory

Remember past chats.

Example:

User: My name is SK

Later: What's my name?

---

5. Tools

Functions/APIs agent can use.

Example:

- Weather API
- News API
- Calculator

---

6. Agents

Smart system that decides which tool to use.

🔥Example

```
llm = ChatOllama(model="llama3.1")


agent = create_agent(
  model=llm,
  tools=[weather_tool]
)
```

🎯 Why Companies Use LangChain

- Fast development
- Production-ready patterns
- Tool integration
- Memory systems
- Agent workflows
- Easy experimentation

⚠️ Limitations

- Can become complex
- Needs good prompt engineering
- Tool errors need handling
- LLM cost/latency

🚀 Better Enhancement

Now many teams also use:

LangGraph

For advanced multi-step agents.

2️⃣ What is LLM?

LLM = **Large Language Model**

A machine learning model trained on massive text data to understand and generate human-like language.

Examples:

- GPT
- Llama
- Claude
- Gemini
- Mistral

🧠 How It Works (Simple)

LLM predicts the next word/token.

Example:

Sky is ___

Predicts:

blue

Repeated billions of times = intelligent responses.

📦 Why “Large”?

Because trained on:

- Huge datasets
- Billions/trillions parameters
- Powerful GPUs

🔥 What LLM Can Do

Text Tasks

- Chat
- Summarization
- Translation
- Rewrite

Coding

- Generate code
- Debug code
- Explain code

Reasoning

- Planning
- Logic
- Multi-step tasks

Real Examples

Student

Explain recursion

Developer

Write Python API

HR

Draft email

Marketer

Create ad copy

⚠️ Limitations

Hallucination

Can generate wrong facts confidently.

No Live Data by Default

Needs tools.

Bias

Depends on training data.

Context Window Limits

Cannot remember everything forever.

3️⃣ What is AI Agent?

AI Agent is an intelligent system that can:

- Understand goals
- Reason
- Decide actions
- Use tools
- Observe results
- Retry
- Complete tasks autonomously

LLM is brain.

Agent is worker.

🧠 Difference Between Chatbot vs Agent

Chatbot

> User asks → reply

Agent

> User asks → think → use tools → decide → finish task

🔥 Example 1: Travel Agent

User says:

> Plan Goa trip under ₹20,000

Agent can:

- Search flights
- Search hotels
- Check weather
- Build itinerary
- Suggest plan

🔥 Example 2: Coding Agent

User says:

> Build login API

Agent can:

- Generate code
- Run tests
- Fix errors
- Create docs

🔥 Example 3: Personal Assistant

User says:

> Schedule tomorrow meeting

Agent can:

- Check calendar
- Find free slot
- Send invite

Core Components of Agent

1. Goal

What user wants.

2. Brain

LLM

3. Tools

APIs/functions

4. Memory

Past context

5. Planning

Next best step

6. Action

Use tool

Agent Flow

User Request

↓

Think

↓

Select Tool

↓

Execute

↓

Observe Result

↓

Final Answer

Why AI Agents Matter

Instead of only generating text, they **do work**.

Popular Agent Frameworks

- LangChain
- LangGraph
- AutoGen
- CrewAI

Challenges

- Hallucination
- Infinite loops
- Tool misuse
- Prompt injection
- Cost
- Latency

Production Enhancements

- Guardrails
- Tool validation
- Retry logic
- Monitoring
- Human approval for risky actions

4️⃣ What is Ollama ?

Ollama is software that lets you run LLM models **locally on your laptop/server**.

Instead of using cloud APIs, models run on your machine.

Why Ollama is Popular

1. Free Local Usage

No per-token API charges.

2. Privacy

Data stays local.

3. Fast Testing

Great for developers.

4. Easy Setup

Simple commands.

Example Commands

ollama pull llama3.1

ollama run llama3.1

Supported Models

- Llama
- Mistral
- Gemma
- Code models

Real Use Cases

Developer Testing

Build chatbot locally.

Offline AI

No internet needed after download.

Company Internal Use

Sensitive data.

Edge Devices

Run on local systems.

How LangChain Uses Ollama

from langchain_ollama import ChatOllama

llm = ChatOllama(model="llama3.1")

Flow:

Your Code → ChatOllama → Ollama → Local Model

Limitations

- Needs RAM / GPU
- Slower than cloud on weak laptops
- Large models need resources

Agent Code Explanation

1️⃣ `ChatOllama` - LLM Integration Layer

Concept

`ChatOllama` is an adapter that lets LangChain communicate with models running inside Ollama.

Why it matters

Your raw model (Llama 3.1) is just an LLM server. LangChain needs a standard interface for:

- sending prompts
- receiving responses
- tool calling support
- message formatting

`ChatOllama` provides that bridge.

Architecture

Python App → ChatOllama → Ollama Runtime → Llama Model

Why developers use this

- Run models locally
- No cloud cost
- Private data
- Fast experimentation

2️⃣ `@tool` — Turning Python Functions into Agent Tools

Concept

The `@tool` decorator converts a normal Python function into something an AI agent can discover and use.

Without `@tool`, your weather function is just code.

With `@tool`, it becomes:

- named capability
- documented function
- callable by the model
- schema-aware input/output tool

Why This is Powerful

Your LLM alone cannot fetch live weather.

But once given a tool:

check_weather(location)

It can reason:

> User wants weather → I should call check_weather

That's the difference between **text generation** and **action-taking AI**.

What the Docstring Does

Example:

"""

Get real-time weather

"""

This is not just documentation.

The model reads it to understand:

- what the tool does
- when to use it

So docstrings become part of tool intelligence.

3️⃣ External APIs as Tools

We have integrated real services:

- Weather API
- News API
- Recipe API

Why This Matters

LLMs have static knowledge. APIs provide:

- real-time data
- trusted sources
- fresh information

This is how modern AI apps are built:

LLM + APIs + Logic

4️⃣ `InMemorySaver()` — Memory / State Management

Concept

Agents need memory. Not just text generation.

`InMemorySaver()` stores:

- conversation history
- previous tool calls
- prior assistant messages
- state across turns

Without memory:

User: My city is Chennai

Later: What's my weather?

Agent forgets city

With memory:

Agent remembers Chennai

Why Called Checkpointer?

Because it saves execution state at checkpoints during the graph workflow.

Useful for:

- resuming conversations
- debugging
- multi-step reasoning

5️⃣ `create_agent()` — Agent Constructor

Concept

This function assembles the full AI system.

You provide:

- Model (brain)
- Tools (hands)
- Prompt (rules)
- Memory (history)

And it creates an executable agent.

What It Builds Internally

Something like:

Input

↓

Read prompt

↓

Understand request

↓

Need tool?

├─ Yes → Call tool

└─ No → Answer directly

↓

Return response

↓

Save memory

This is often implemented as a graph/state machine.

6️⃣ `system_prompt` — Hidden Operating Manual

Concept

The system prompt defines agent behavior before the user speaks.

It controls:

- personality
- tool usage rules
- safety boundaries
- output formatting
- anti-hallucination behavior

Why This Is Critical

Same model + different prompt = different behavior.

Your prompt teaches:

- use only valid tools
- don’t hallucinate
- don’t leak secrets
- use one tool when relevant
- be concise

That’s production thinking.

7️⃣ `graph.invoke()` — Execute One Agent Run

Concept

`invoke()` means:

> Run the agent once with this input.

It triggers the full reasoning cycle.

Internally It Does

Receive message

↓

Load memory by thread_id

↓

Analyze request

↓

Select tool if needed

↓

Execute tool

↓

Observe result

↓

Generate final answer

↓

Save memory

↓

Return structured response

Why It Returns Messages

Because agents are chat systems, not single strings.

Response often includes:

- user message
- tool calls
- tool outputs
- final assistant reply

You print the last assistant message.

8️⃣ `thread_id` — Session Identity

Concept

This identifies one conversation.

Examples:

thread_id = user_101

thread_id = customer_A

thread_id = mobile_session_xyz

Each thread gets isolated memory.

Without it, users share memory accidentally.

9️⃣ Structured Messages Format

You used:

```
{
"messages": [
  {"role": "user", "content": "..."}
]
}
```

Why Important

Modern LLMs think in conversations:

- system
- user
- assistant
- tool

This format preserves roles and context.

🔟 Reasoning + Tool Use = Agentic Behavior

Your system is not simple Q&A.

It performs:

Interpret intent

Choose capability

Execute action

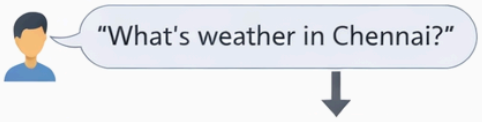
Use result

Respond naturally

That’s the foundation of AI agents.

Full Agent Flow

Example Query: "What's weather in Chennai?"



1. User Query

- *What's weather in Chennai?"*

2. Read Agent Rules

Access System Prompt

- *Use Tools*
- *No Hallucination*
- *Valid Actions Only*

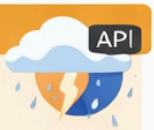
3. Understand Intent

Detects: Weather Request

4. Select Tool

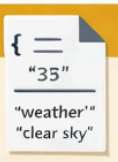
Call: check_weather("Chennai")

5. Fetch Weather Data



- *API Request to OpenWeather*
- *Live Data Retrieval*

6. Process JSON



- **"location": "Chennai",**
- **"temperature": 32,**
- **"weather": "clear sky"**

7. Generate Response

"Current weather in Chennai is 32°C with sky."

8. Save to Memory

- *Store: **thread_id = 1***
- *Conversation Context*

9. Wait for Next Query